The models inside ClearRun

There are three models. The invoice scanner uses AI to read documents. The customer risk model and the cash shortfall simulator are statistical models: transparent, repeatable and explainable line by line.

Version 5 · October 2026

AI + rules

1. Invoice scanner

Turns a photo or PDF of an invoice into checked fields: number, customer, GSTIN, amount and dates.

Statistical

2. Customer risk model

Scores each customer 0 to 100 from how they actually paid, and recommends credit terms for the next order.

Statistical

3. Cash shortfall simulator

Runs 1,500 versions of the next 13 weeks to estimate the chance that cash drops below the reserve or below zero.

1. Invoice scanner

Job: remove the slowest step, typing invoices in, without letting a misread number reach the forecast. The AI model only *reads* the invoice. Plain rules then check everything it reads.

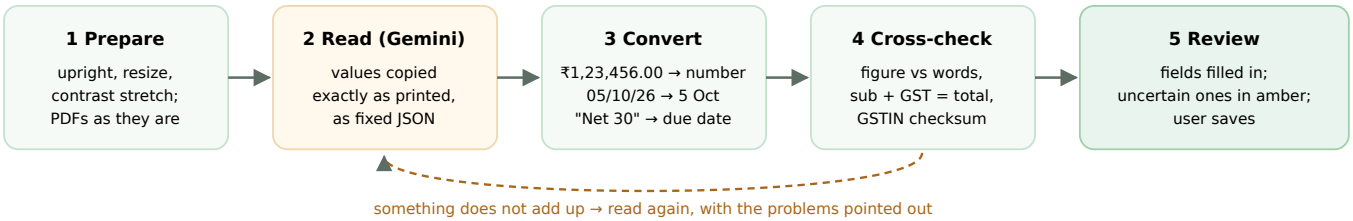


Figure 1. Scanner pipeline. Only step 2 uses AI. Steps 3 and 4 are fixed rules that give the same result every time.

The checks

Check	What it catches
Total vs amount in words	The words ("Rupees Two Lakh Thirty Six Thousand Only") are converted to a number, including lakh, crore and paise, and compared with the figure. A mismatch means one of them was misread.
Subtotal + GST + round off = total	Catches the most common mistake: taking the taxable value, before GST, as the invoice total.
GSTIN checksum	The 15th character of a GSTIN is a check character calculated from the other 14. A GSTIN that fails the check is left empty rather than saved wrong.
Buyer vs seller	If the customer's name or GSTIN matches the seller's, the two were probably swapped. The "Bill To" party is the customer.
Date sense	Dates are read day first, as in India. A date that could be either way is flagged for the user. Dates in the future, more than 3 years old, or a due date before the invoice date are also flagged.
Payment terms	If no due date is printed, "Net 30" or "within 45 days" is used to work it out, and the user is told.

Second reading. If a check fails or key fields are missing, the invoice is read again with the specific problems listed. The reading that passes more checks is kept, and gaps are filled from the other reading

but marked for review.

Confidence. Each field is marked high, medium or low, based on the model's own rating adjusted by the checks. Medium and low fields are highlighted in amber, so the user looks where it matters.

Staying available. If Google's model is busy or retired, the scanner moves to the next model in the list: Gemini 3.8 Flash, then 3.6 Flash, then 3.5 Flash-Lite. Each attempt gets at most 18 seconds, and the whole scan finishes within about 50 seconds, so the user always gets an answer.

Honest limits.
The rules were tested on prepared readings: 11 automated tests, covering a misread total, swapped parties and recovery by the second reading. Accuracy has not been measured on a large set of real invoices. Handwritten bills and very blurred photos will need more correcting by hand. The user always reviews the fields before saving.

2. Customer risk model

Job: answer "should I give this customer credit, and on what terms?" using how they actually paid. It does not use machine learning, so every point in a score can be explained. This is the team's Python model rebuilt in JavaScript with identical rules. On a shared test set of 18 customers, every decision matched and every score was within one point.

Six payment signals (score 0 to 100, higher is riskier)

Signal	Measured as	Points
Lateness	Typical (median) days late on paid invoices	0 days = 0 … 45+ days = 25
Unpredictability	Bad case (80th percentile) minus typical delay	0 days = 0 … 30+ days = 15
Very late	Share of invoices paid, or still unpaid, 45+ days late	up to 20
Broken promises	Share of payment promises broken (30% assumed if fewer than 2 promises recorded)	up to 15
Overdue now	Oldest unpaid invoice, days past due	0 days = 0 … 60+ days = 15
Slowing down	Last 4 payments 7+ days later than usual (needs 6+ paid invoices)	10

0 – 30
Full credit
Give the terms they ask for.

31 – 55
Limited credit
Shorter terms, part advance or a dated cheque, and a credit limit.

56 – 100
Advance only
50 to 100% before dispatch.

New customers: compare with similar customers

A new customer has no history, so the model finds the 5 most similar customers that have at least 5 paid invoices. It averages their scores and delays, giving more weight to the closer ones.

```
distance = 3.0 × (type differs)
         + 1.5 × |ln(1+years_a) − ln(1+years_b)| / ln 31
         + 1.0 × |ln(order_a) − ln(order_b)| / ln 100
         + 1.0 × |terms_a − terms_b| / 60
weight   = 1 / (distance + 0.1)
```

Confidence reflects how much those 5 neighbours agree: high if at least 80% fall in the same class. A customer with a short history gets a blend of their own score and their neighbours' score. The weight on their own history is `n / (n + 5)` for *n* paid invoices, rising to 100% at 8 invoices. Problems happening right now, such as overdue invoices, always count in full.

Deal advice

```
credit limit = factor × monthly billing × max(1, (credit days + typical delay) / 30)
factor: Full 1.5 · Limited 1.0 · Advance 0.5
```

Recommended actions follow from *why* a customer is risky. If they often break promises, the advice is a dated cheque instead of a verbal promise. If their timing varies widely, the advice is to split the delivery. Overdue money must be collected before dispatch, and a slowing payer should get a call before their credit is raised. The model also predicts when the money will really arrive (due date plus typical delay), and the user can add that payment to the forecast in one click.

Where the history comes from. ClearRun builds it from the user's own invoices. Days late are the paid date minus the due date. Overdue days and amounts owed come from unpaid invoices past their due date. Older history can also be typed in.

3. Cash shortfall simulator (Monte Carlo)

Job: a forecast with a single line hides the chance of a bad week. The simulator replays the next 13 weeks 1,500 times with realistic variation in *when* customer money arrives, then counts how often cash falls too low.

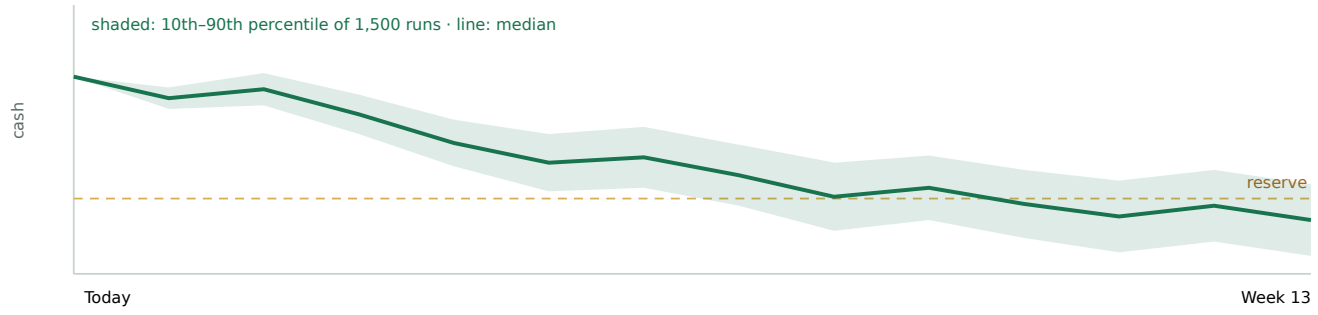


Figure 2. Illustration of the output: a range of likely cash positions instead of a single line.

Step	What happens in each of the 1,500 runs
Customer receipts	Each expected receipt lands in its planned week, shifted by a random amount drawn from a normal distribution with a standard deviation of 0.6 weeks and rounded to whole weeks. Receipts that drift past week 13 are left out.
Scenario: delay	The "delay a customer payment" slider moves this week's receipts later by 0 to 6 weeks.
Scenario: sales	The sales slider scales receipts from week 3 onward, from −50% to +20%.
Scenario: shock	An unexpected expense is taken out in week 1.
Payments	Bills go out in their planned weeks, without random variation.

Outputs: the chance of falling below the reserve, the chance of negative cash, the 10th, 50th and 90th percentile balance for each week, the average lowest balance, and the typical week of the first shortfall. A fixed random seed is calculated from the inputs, so the same inputs always give the same answer. A presentation can therefore be repeated exactly.

Honest limits.
The ±0.6-week timing variation is the same for every customer, because few accounts have payment history yet. The natural next step is to use each customer's typical and bad-case delay from the risk model, so that late payers widen the range more than reliable ones.

How the three models work together

Scanner → data
Scanned invoices become expected receipts in the forecast, and once paid they become payment history.

History → risk
Paid and overdue invoices feed each customer's score, and the score sets terms for the next deal.

Plans → simulation
Expected receipts, bills and scenarios feed the simulator, which turns them into the chance of a shortfall.